第3课：卷积神经网络

- 李伟昌, weichangli@zju.edu.cn
- 浙江大学**海洋人工智能**课程讲义
- 版本 1, 2026年5月20日

时长： 45分钟
目标： 理解CNN并将其应用于计算机视觉任务

3.1 动机：全连接有什么问题？

MLP处理图像的问题

考虑一个小图像：224 × 224 × 3 = 150,528像素

第一层到1000个神经元：**1.5亿参数！**

问题：

- 参数太多 → 过拟合
- 忽略空间结构（相距较远的像素无关）
- 没有平移不变性

神经科学的关键见解

Hubel & Wiesel（1960年代）发现：

- 简单细胞：对特定方向的边缘有反应
- 复杂细胞：对模式有反应，无论确切位置

CNN在计算上实现了这些想法。

3.2 卷积操作

直观理解

在图像上滑动小滤波器（核），在每个位置计算点积。

2D卷积公式

给定输入$\mathbf{X}$和核$\mathbf{K}$：

$$(\mathbf{X} * \mathbf{K})_{i,j} = \sum_m \sum_n \mathbf{X}_{i+m,j+n} \cdot \mathbf{K}_{m,n}$$

可视化演练：卷积操作

输入图像（5×5）： 核（3×3）： 输出（3×3）：

```
┌───┬───┬───┬───┬───┐        ┌───┬───┬───┐
│ 1 │ 1 │ 1 │ 0 │ 0 │        │ 1 │ 0 │ 1 │
├───┼───┼───┼───┼───┤        ├───┼───┼───┤
│ 0 │ 1 │ 1 │ 1 │ 0 │   *    │ 0 │ 1 │ 0 │   =   [逐步见下文]
├───┼───┼───┼───┼───┤        ├───┼───┼───┤
│ 0 │ 0 │ 1 │ 1 │ 1 │        │ 1 │ 0 │ 1 │
├───┼───┼───┼───┼───┤        └───┴───┴───┘
│ 0 │ 0 │ 1 │ 1 │ 0 │
├───┼───┼───┼───┼───┤
│ 0 │ 1 │ 1 │ 0 │ 0 │
└───┴───┴───┴───┴───┘
```

第1步： 将核放在左上角，计算点积：

```
(1×1 + 1×0 + 1×1) + (0×0 + 1×1 + 1×0) + (0×1 + 0×0 + 1×1)
= 1 + 0 + 1 + 0 + 1 + 0 + 0 + 0 + 1 = 4
```

第2步： 将核向右滑动（stride=1），计算：

```
(1×1 + 1×0 + 0×1) + (1×0 + 1×1 + 1×0) + (0×1 + 1×0 + 1×1)
= 1 + 0 + 0 + 0 + 1 + 0 + 0 + 0 + 1 = 3
```

最终输出：

```
┌───┬───┬───┐
│ 4 │ 3 │ 4 │
├───┼───┼───┤
│ 2 │ 4 │ 3 │
├───┼───┼───┤
│ 2 │ 3 │ 4 │
└───┴───┴───┘
```

关键超参数

参数	描述	常见值
核大小（k）	滤波器大小	3×3, 5×5, 7×7
步幅（s）	滑动步长	1, 2
填充（p）	边界添加的零	0, 1, k//2
输出通道	滤波器数量	32, 64, 128, 256

输出大小公式

对于输入大小W，核K，填充P，步幅S：

$$W_{out} = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

常见模式：

- "Same"卷积：$P = (K-1)/2$，步幅1 → 输出 = 输入大小
- "Valid"卷积：$P = 0$ → 输出缩小
- 下采样：步幅2 → 空间维度减半

示例：边缘检测

```
import torch
import torch.nn as nn

# Sobel边缘检测核
sobel_x = torch.tensor([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=torch.float32)
sobel_y = torch.tensor([[-1, -2, -1], [0, 0, 0], [1, 2, 1]], dtype=torch.float32)

# 在PyTorch中
conv = nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3, padding=1)
```

卷积特性

1. **稀疏连接：** 每个输出仅依赖$k \times k$输入
2. **参数共享：** 相同的滤波器应用于所有位置（平移等变）
3. **平移等变：** 输入移位 → 输出移位（但检测到模式）

3.3 CNN构建模块

卷积层

```
nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, stride=1, padding=1)
```

参数： $C_{out} \times C_{in} \times K \times K$

激活函数

ReLU在卷积后逐元素应用：

$$\mathbf{A} = \text{ReLU}(\mathbf{Z}) = \max(0, \mathbf{Z})$$

池化层

最大池化：

- 取每个$k \times k$窗口的最大值
- 提供平移不变性
- 减少空间维度

```
nn.MaxPool2d(kernel_size=2, stride=2)  # H和W减半
```

平均池化：

- 取平均值；更平滑的下采样
- 常用于最后层

趋势： 现代架构使用步幅卷积代替池化。

ConvNet的批归一化

```
nn.BatchNorm2d(num_features=64)
```

跨批次和空间维度归一化：每个通道独立归一化。

CNN特征学习可视化

每层学习什么（层次特征提取）：

```
第1层（32个滤波器）：        第3层（128个滤波器）：       第5层（512个滤波器）：

┌───┬───┬───┬───┐           ┌────┬────┬────┐              ╔════╗
│ / │ — │ ╲ │ │ │           │ ○○ │ ▓▓ │ ▫▫ │              ║ 🐱 ║
│ / │ — │ ╲ │ │ │           │ ○○ │ ▓▓ │ ▫▫ │              ║ 🐶 ║
└───┴───┴───┴───┘           └────┴────┴────┘              ╚════╝
  边缘                         纹理                          物体
（水平、                      （圆形、                       （面部、
垂直、                        网格、                         汽车、
对角）                        角点）                         动物）
```

典型ConvNet架构流程：

```
输入图像（3×224×224）
    ↓
Conv2d(3→64, 3×3) + ReLU  ──→ 特征图（64×224×224）
    ↓
MaxPool2d(2×2)            ──→ (64×112×224)  [空间缩减]
    ↓
Conv2d(64→128, 3×3) + ReLU ──→ (128×112×112)
    ↓
MaxPool2d(2×2)            ──→ (128×56×56)
    ↓
Conv2d(128→256, 3×3) + ReLU ──→ (256×56×56)
    ↓
全局平均池化               ──→ (256×1×1)
    ↓
全连接(256→10)             ──→ 类别分数
    ↓
Softmax                    ──→ 概率
```

关键模式：

- 卷积层提取特征（边缘 → 纹理 → 图案 → 物体）
- 池化减少空间维度
- 深度（通道）增加，空间维度减少
- 最后层做出分类决策

3.4 经典CNN架构

LeNet-5 (1998) - LeCun等人

先驱：

- 2个卷积层
- 2个池化层
- 2个全连接层
- ~60K参数
- 应用于MNIST

AlexNet (2012) - Krizhevsky等人

突破：

- 8层（5卷积 + 3全连接）
- ReLU激活
- Dropout正则化
- 数据增强
- GPU训练
- 60M参数

为什么重要： 赢得ImageNet 2012，领先10+百分点。

VGGNet (2014) - Simonyan & Zisserman

关键见解：**小(3×3)卷积的堆叠** ≈ 更大感受野，参数更少

- VGG-16：13卷积 + 3全连接
- VGG-19：16卷积 + 3全连接
- 138M参数
- 简单、统一的架构

感受野计算：

- 3×3卷积，步幅1：每层感受野增加2
- 三个3×3层 ≈ 一个7×7层
- 但参数为3×(3^2C^2) = $27C^2$ vs 7^2C^2 = $49C^2$

ResNet (2015) - He等人

革命：**跳跃连接**解决梯度消失

残差块：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$$

学习残差函数$\mathcal{F}$而不是直接映射。

变体：

- ResNet-18：18层
- ResNet-34：34层
- ResNet-50：50层（瓶颈块）
- ResNet-101/152：更深

瓶颈块：

```
1×1卷积（降维）→ 3×3卷积 → 1×1卷积（扩展）
```

减少计算同时保持深度。

```
# 基本残差块
class BasicBlock(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_ch)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_ch)

        self.shortcut = nn.Sequential()
        if stride != 1 or in_ch != out_ch:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 1, stride, bias=False),
                nn.BatchNorm2d(out_ch)
            )

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += self.shortcut(x)
        out = F.relu(out)
        return out
```

现代架构（简要概述）

EfficientNet (2019)：

- 复合缩放：深度、宽度、分辨率一起缩放
- EfficientNet-B0到B7（B7：66M参数，最先进效率）

ConvNeXt (2022)：

- 现代化的纯CNN，与Vision Transformers竞争
- 更大的核（7×7），更少的激活，LayerNorm

3.5 迁移学习

动机

- 从头训练：数百万图像，数天计算
- 大多数数据集很小
- 早期层学习通用特征（边缘、纹理）
- 后期层学习任务特定特征

迁移学习策略

策略1：特征提取

- 冻结预训练层
- 只训练最终分类器
- 快速，小数据过拟合少

策略2：微调

- 用预训练权重初始化
- 训练所有（或部分）层
- 预训练层使用较低学习率
- 数据充足时性能更好

```
import torchvision.models as models

# 加载预训练ResNet
model = models.resnet18(pretrained=True)

# 策略1：冻结主干
for param in model.parameters():
    param.requires_grad = False

# 替换最后层
model.fc = nn.Linear(model.fc.in_features, num_classes)

# 策略2：使用不同学习率微调
optimizer = optim.Adam([
    {'params': model.fc.parameters(), 'lr': 1e-3},
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.layer1.parameters(), 'lr': 1e-5}
])
```

常见预训练模型

- ImageNet（最常见）
- Places365（场景识别）
- 医学影像（ChestX-ray14等）

3.6 视觉任务概述

图像分类

输入：图像 → 输出：类别标签

目标检测

输入：图像 → 输出：边界框 + 类别标签

- 两阶段：R-CNN系列
- 单阶段：YOLO、SSD

语义分割

输入：图像 → 输出：逐像素类别标签

- FCN、U-Net、DeepLab

实例分割

分割单个物体实例（Mask R-CNN）

3.7 实验：CNN实践

练习1：实现简单CNN

构建并在CIFAR-10上训练CNN：

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),  # 32x32 -> 16x16
            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),  # 16x16 -> 8x8
            nn.Conv2d(64, 128, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),  # 8x8 -> 4x4
        )
        self.classifier = nn.Sequential(
            nn.Linear(128 * 4 * 4, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, 10)
        )

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)  # 展平
        x = self.classifier(x)
        return x
```

练习2：比较架构

在CIFAR-10上比较：

- 简单CNN（从头开始）
- ResNet18迁移学习（冻结）
- 微调的ResNet18

练习3：特征可视化

可视化不同层学习的内容：

- 第一层滤波器（边缘、颜色）
- 中间层激活
- 最终特征的t-SNE

关键要点

1. CNN通过局部连接和参数共享利用空间结构
2. 三个关键操作：卷积、激活、池化
3. 架构演进：LeNet → AlexNet → VGG → ResNet → EfficientNet
4. 跳跃连接（ResNet）支持训练非常深的网络
5. 迁移学习允许利用预训练模型处理新任务
6. 现代CNN可以与Vision Transformers竞争或互补

延伸阅读

核心材料

1. CS231n CNN Notes
2. Deep Residual Learning Paper
3. A Recipe for Training Neural Networks - Andrej Karpathy
4. An Interactive Node-Link Visualization of CNNs - CNN的交互式可视化

视频资源

5. **Andrej Karpathy的Neural Networks: Zero to Hero** - 第5讲涵盖CNN和现代计算机视觉。这些笔记中的层次特征可视化借鉴了Karpathy关于层如何从边缘到物体逐步构建的解释。

交互工具

6. **CNN Explainer** - 一个交互式可视化系统，旨在帮助非专家学习卷积神经网络（CNN）。使用此工具可视化卷积和池化操作如何逐层转换输入图像。